

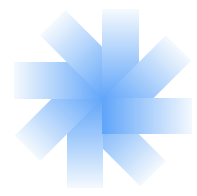


SOLARPULSE

SOLAR PULSE

Solar Energy Underperformance Detection System

Presented by: Aditya Ujjwal



SOLARPULSE

Project Overview



The Problem

Raw solar generation data alone does not reveal whether output is normal or abnormally low. Manual monitoring fails to catch persistent underperformance, hinders cross-site comparison and prioritization, and provides no insight into potential energy loss.

The Solution

SolarPulse combines MySQL (structured storage), Python + Pandas (cleaning and feature engineering), Machine Learning (expected generation prediction), performance analysis (actual vs expected), Power BI (interactive monitoring), and Streamlit (interactive prediction app) into one end-to-end pipeline.

End-to-End Data Flow

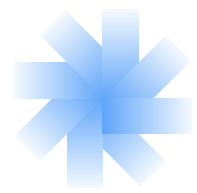
RAW SOLAR DATA → SQL → PYTHON CLEANING → MACHINE LEARNING → UNDERPERFORMANCE DETECTION → POWER BI + STREAMLIT

Business Value

"SolarPulse transforms raw solar data into actionable information about expected production, performance gaps, potential energy loss, and sites requiring investigation."

Project Description

An end-to-end solar energy analytics and ML system analyzing historical generation and weather data to estimate expected generation and detect underperformance across all monitored sites.



SOLARPULSE

Technology Stack



Storage Layer

MySQL

Processing Layer

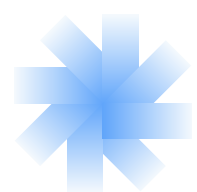
Python &
Pandas

ML & Analysis

Scikit-learn ·
Matplotlib · Seaborn

Application Layer

Power BI · Streamlit



SOLARPULSE



UNISOLAR Dataset

solar_sites

42

generation_raw

2.73M

weather_raw

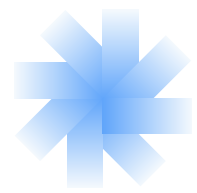
371,769

monthly_summary

1,176

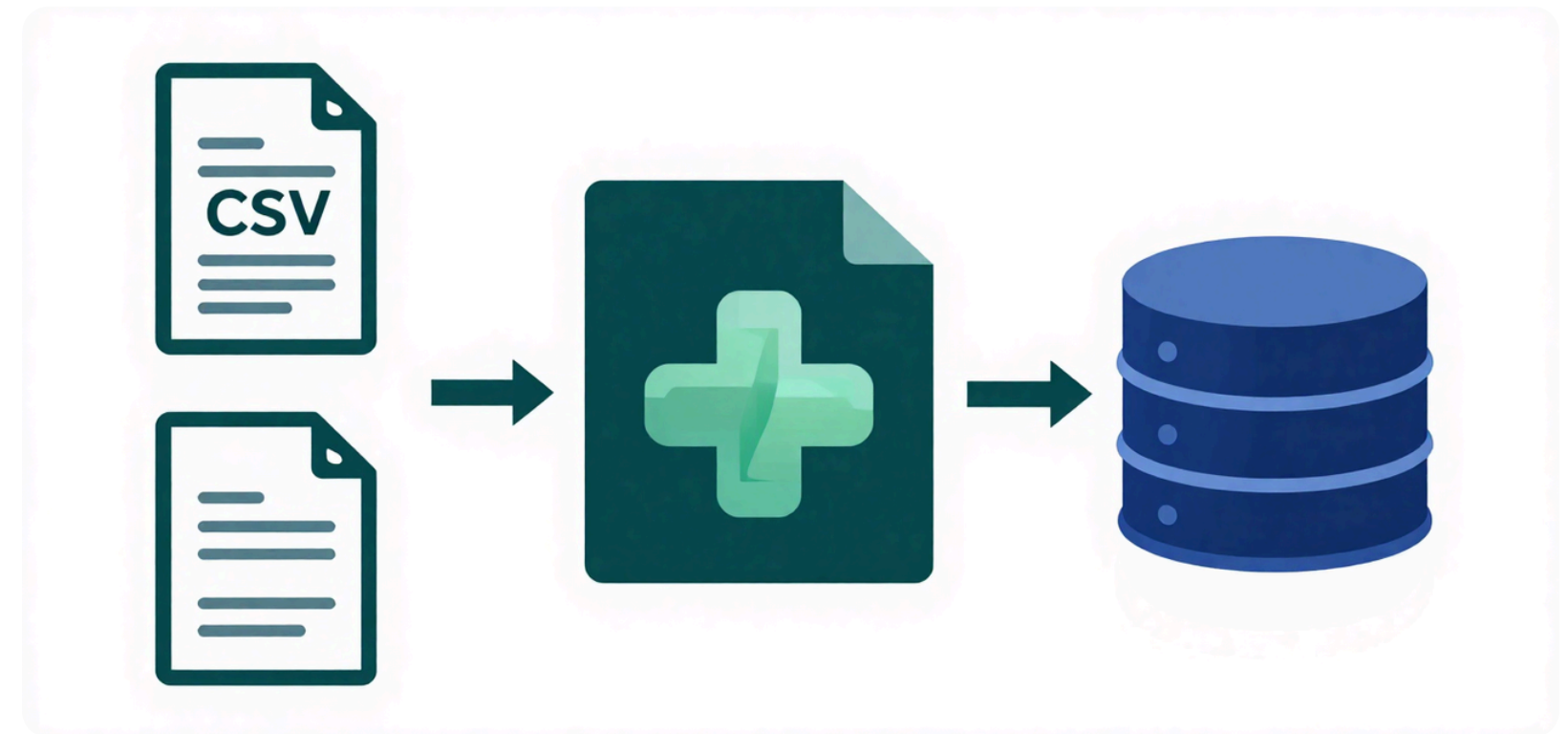
Total Raw Records

3.1M+



SOLARPULSE

Process 1 MySQL Database & Data Loading



LOADING PIPELINE

Four CSV source files are imported via a Python script using Pandas controlled import with chunked loading for large tables. Index management ensures query performance. Each table's row count is verified post-import.

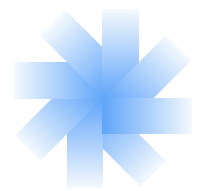
WHY MYSQL

Scalable relational storage designed for millions of records, structured querying, and future data growth. Supports SQL audit, analytical views, and direct Power BI connection.

VALIDATED ROW COUNTS

- solar_sites — 42 records
- monthly_summary — 1,176 records
- weather_raw — 371,769 records
- generation_raw — 2,731,946 records

Total: >3.1 million raw records loaded across four structured tables. Large tables imported in chunks with index management for performance and integrity.



SolarPulse

	total_rows	valid_generation	missing_generation
▶	2731946	1195645	1536301

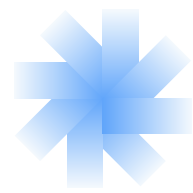
Result Grid	Filter Rows:	Export:	Wrap Cell Content:
SiteKey	total_records	missing_records	missing_percentage
▶ 7	79319	59127	74.54
5	47215	29246	61.94
9	79319	48325	60.92
35	72604	43371	59.74
29	60403	35723	59.14
28	59948	35436	59.11
13	77303	45673	59.08
4	47215	27738	58.75
16	47124	27577	58.52
34	72316	41941	58.00
21	64252	36138	56.24
32	64252	36102	56.19
39	58700	32934	56.11

Process 2 — SQL Data Quality Audit

Generation: 2,731,946 total rows | 1,195,645 valid | 1,536,301 missing (56.23%) | 0 negative | 0 invalid

Weather missingness: AirTemp, ApparentTemp, DewPoint, RelativeHumidity — 107,113 missing each; WindSpeed & WindDirection — 162,890 missing each.

Timestamps at 15-min intervals; alignment verified. Missingness concentrated outside production hours — not random. No invalid humidity or wind-direction values detected.



SOLARPULSE

What the Audit Revealed

01

56.23% generation missing → Do not replace missing values with zero; use observed targets only.

02

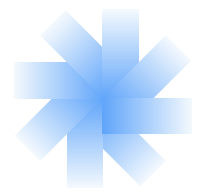
Missingness outside production hours → Train supervised model on observed targets only; preserve missing records.

03

Weather missingness substantial → Targeted imputation; interpolate short gaps only; avoid long-gap interpolation.

04

No negative generation or invalid values → Retain valid observations; no unnecessary removals or fabrication.



SOLARPULSE

Process 3 — Data Cleaning with Python & Pandas

Cleaning Workflow Steps:

1. Load raw MySQL data (generation, weather, site, monthly tables)
2. Inspect structure, column types, and data shapes
3. Convert and standardize timestamps to datetime
4. Check and remove duplicate records
5. Validate physical ranges (generation, humidity, wind speed)
6. Analyze missingness patterns across all tables
7. Handle weather missing values via interpolation and median fallback

Dataset Integration & Output:

1. Preserve missing generation indicators (do not fill with zero)
2. Align generation and weather on Campus + Timestamp
3. Merge generation, weather, site metadata, and time features
4. Engineer cyclical and time features for ML
5. Filter to observed generation targets only
6. Export final ML-ready dataset

Before → After: Raw, incomplete, misaligned tables → validated, aligned, feature-engineered ML dataset ready for Random Forest training.

```
FINAL ML DATASET
=====
Rows: 1,181,498
Columns: 40

Missing target: 0
Missing weather: 0

Sites: 42
Campuses: 5
```

```
Monthly Summary Missing Values
```

	Missing_Count
SiteKey	0
Year	0
DataStatus	0
AverageSolarGeneration	200
MaxSolarGeneration	200
MinSolarGeneration	200

```
Site Metadata Missing Values
```

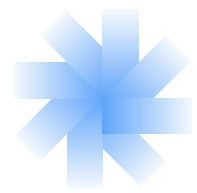
	Missing_Count
CampusKey	0
SiteKey	0
kWp	17
Number_of_Panels	17
Panel	17
Inverter	17
Optimizers	19
Metric	17
Latitude	0
Longitude	0

```
Missing_Count
```

CampusKey	0
SiteKey	0
Timestamp	0
SolarGeneration	1536301

```
Weather Missing Values
```

	Missing_Count
CampusKey	0
Timestamp	0
ApparentTemperature	107113
AirTemperature	107113
DewPointTemperature	107113
RelativeHumidity	107113
WindSpeed	162890
WindDirection	162890



SOLARPULSE

Missing Value Treatment

01

Generation: Missing SolarGeneration is preserved as-is. No zero-fill. Only observed targets used for supervised ML training.

02

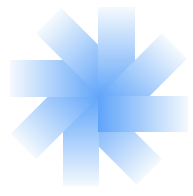
Fallback strategy: Final missing weather values filled with Campus-level median. No fabricated weather rows added.

03

Weather: Missing indicators preserved. Short gaps interpolated. Longer gaps filled via Campus + Month + Hour median.

04

Rationale: Short gaps are reasonably bridged. Long gaps are not linearly interpolated to avoid unrealistic values.



SOLARPULSE

Feature Engineering & Outlier Strategy

Create rolling historical generation

```
ml_data["RollingGeneration_1Hour"] = (  
    ml_data  
    .groupby("SiteKey")["SolarGeneration"]  
    .transform(  
        lambda x:  
            x.shift(1)  
            .rolling(  
                window=4,  
                min_periods=1  
            )  
            .mean()  
    )  
)
```

Create better time representation

```
ml_data["Hour_sin"] = np.sin(  
    2 * np.pi * ml_data["Hour"] / 24  
)  
  
ml_data["Hour_cos"] = np.cos(  
    2 * np.pi * ml_data["Hour"] / 24  
)  
  
ml_data["DayOfYear_sin"] = np.sin(  
    2 * np.pi *  
    ml_data["DayOfYear"] / 365.25  
)  
  
ml_data["DayOfYear_cos"] = np.cos(  
    2 * np.pi *  
    ml_data["DayOfYear"] / 365.25  
)
```

Keep wind direction properly encoded

```
ml_data["WindDirection_sin"] = np.sin(  
    np.radians(ml_data["WindDirection"])  
)  
  
ml_data["WindDirection_cos"] = np.cos(  
    np.radians(ml_data["WindDirection"])  
)
```

Time Features

Hour
DayOfWeek
Month
DayOfYear
IsWeekend

Cyclical Encoding

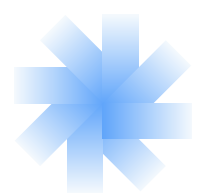
Hour_sin/cos
DayOfYear_sin/cos
(Captures circular time patterns)

Wind Direction

WindDirection_sin
WindDirection_cos
(Handles 0°=360° circularity)

Outlier Strategy

Data-quality: impossible values →
correct/remove/flag
Operational anomalies: plausible
unusual generation → keep &
investigate



SOLARPULSE

Final Machine Learning Dataset

Timestamp	SolarGeneration	AirTemperature	RelativeHumidity	WindSpeed	Hour_sin	Hour_cos	DayOfYear_sin	DayOfYear_cos	WindDirection_sin	WindDirection_cos	PreviousGeneration	Generation_1HourAgo
2020-01-01 06:15:00	0.135	15.247	67.000	5.747	1.000	0.000	0.017	1.000	0.124	-0.992	NaN	NaN
2020-01-01 06:30:00	0.465	15.213	67.267	6.427	1.000	0.000	0.017	1.000	0.092	-0.996	0.135	NaN
2020-01-01 06:45:00	1.039	15.160	68.000	6.867	1.000	0.000	0.017	1.000	-0.167	-0.986	0.465	NaN
2020-01-01 07:00:00	1.673	15.547	67.000	8.293	0.966	-0.259	0.017	1.000	0.014	-1.000	1.039	NaN
2020-01-01 07:15:00	2.560	15.920	66.000	7.640	0.966	-0.259	0.017	1.000	-0.125	-0.992	1.673	0.135
2020-01-01 07:30:00	3.553	16.560	63.933	9.880	0.966	-0.259	0.017	1.000	-0.014	-1.000	2.560	0.465
2020-01-01 07:45:00	4.647	17.233	61.400	7.467	0.966	-0.259	0.017	1.000	0.394	-0.919	3.553	1.039
2020-01-01 08:00:00	5.934	17.753	59.333	7.133	0.866	-0.500	0.017	1.000	0.448	-0.894	4.647	1.673
2020-01-01 08:15:00	7.113	18.167	57.867	6.427	0.866	-0.500	0.017	1.000	-0.022	-1.000	5.934	2.560
2020-01-01 08:30:00	8.485	18.840	56.200	5.893	0.866	-0.500	0.017	1.000	0.306	-0.952	7.113	3.553

DATASET ASSEMBLY

Generation + Weather + Site + Time features are merged into one integrated dataset. Rows without an observed SolarGeneration target are excluded from supervised training — only real measured values are used.

FEATURES INCLUDED

- SITE: CampusKey, SiteKey
- WEATHER: AirTemperature, ApparentTemperature, DewPointTemperature, RelativeHumidity, WindSpeed, WindDirection_sin, WindDirection_cos
- TIME: Hour_sin, Hour_cos, DayOfYear_sin, DayOfYear_cos, DayOfWeek, Month, IsWeekend
- TARGET: SolarGeneration

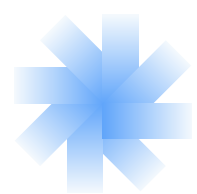
WHY FILTER TO OBSERVED TARGETS?

Missing generation values (56.23% of records) are predominantly outside production hours — replacing them with zero would corrupt the model. Only rows with real observed generation are used for training.

CYCLICAL ENCODING

Time features (Hour, DayOfYear) and WindDirection are encoded as sine/cosine pairs to preserve their circular nature — ensuring the model understands that Hour 23 is close to Hour 0, not far from it.

Historical lag features were explored experimentally but not selected for the primary model to avoid data leakage across sites.



SOLARPULSE

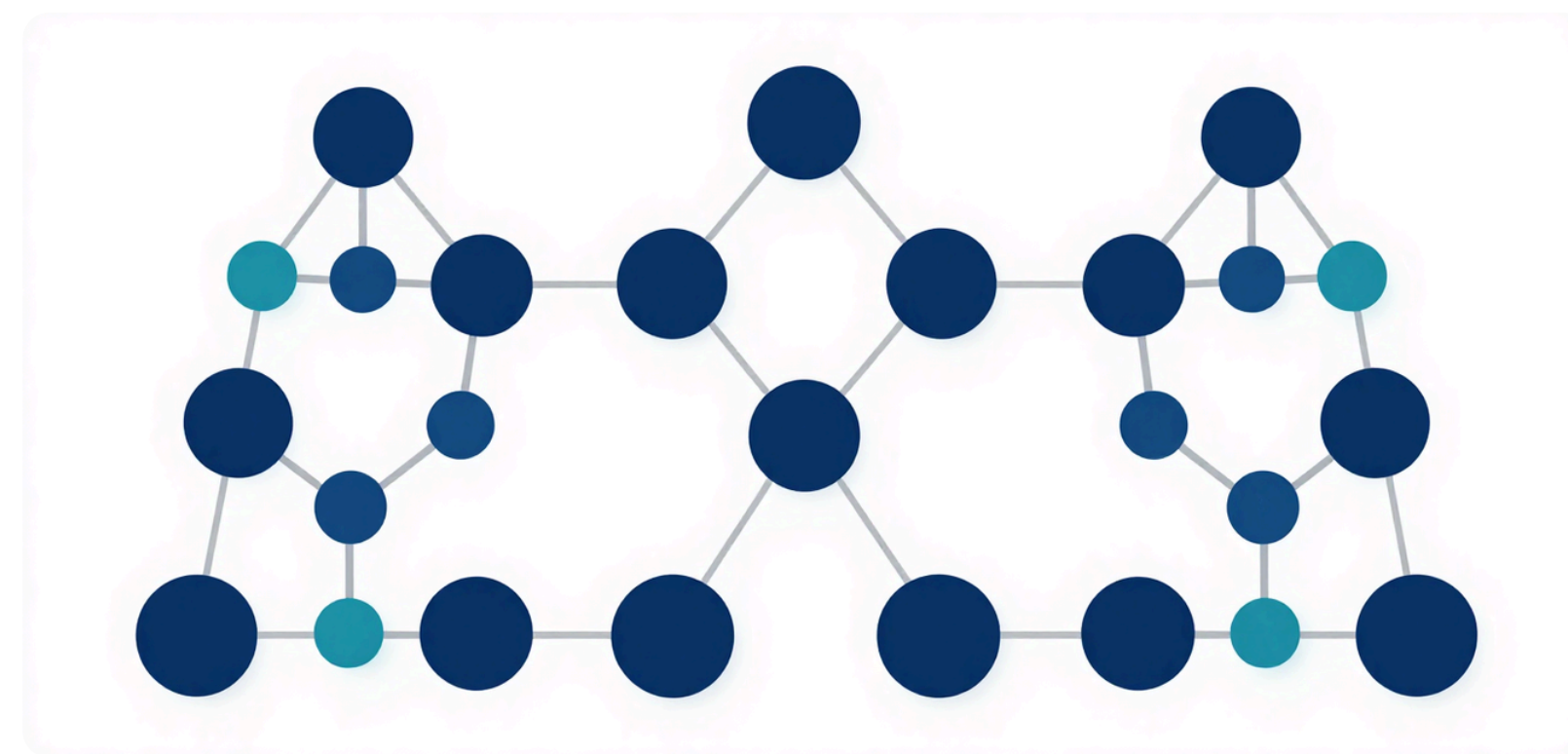
Process 4 — Machine Learning Model

Random Forest regression estimates expected solar generation under given site, weather, and temporal conditions. As an ensemble of decision trees, it captures nonlinear relationships between inputs and output — essential for solar generation patterns that vary by time of day, season, site capacity, and weather simultaneously.

Model Input Flow:

Site Features + Weather Features + Time Features → Random Forest Regressor → Expected Solar Generation

Target variable: SolarGeneration (observed, non-missing values only).



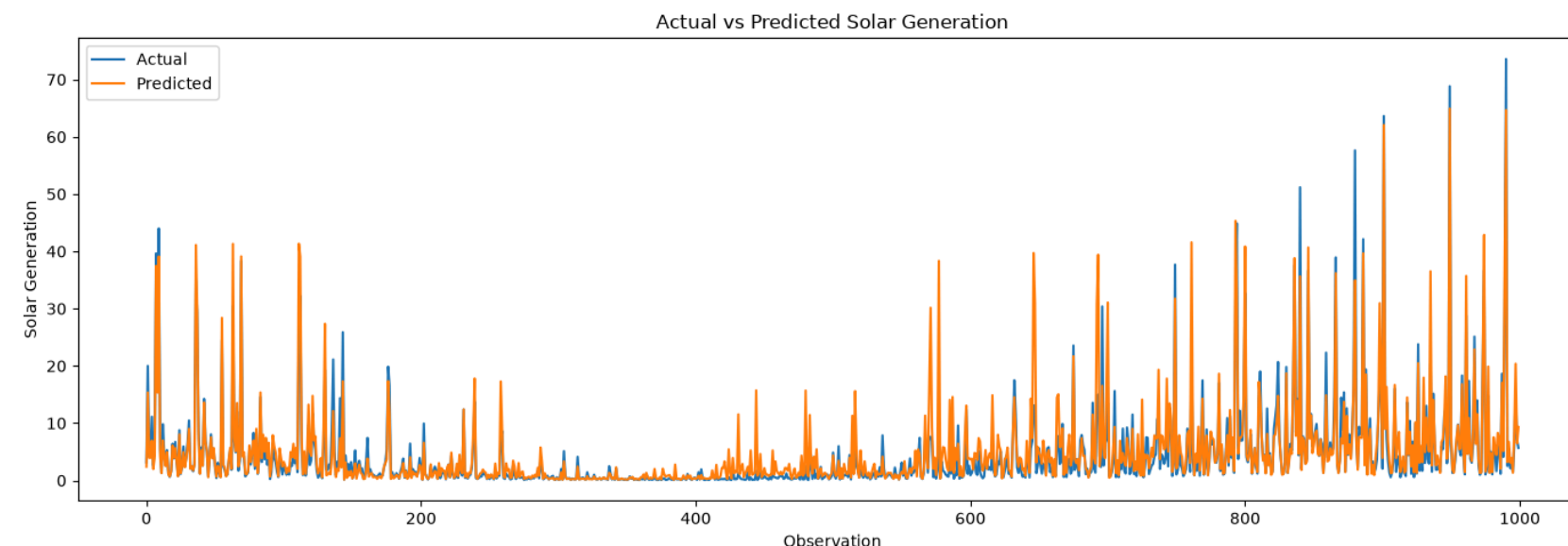
Why Random Forest?

- Handles nonlinear and complex interactions between temporal, weather, and site inputs without manual feature crosses
- Accepts mixed feature types with limited preprocessing requirements
- Provides built-in feature importance to understand which inputs drive expected generation
- Robust to outliers and noise in large tabular datasets
- Scales well to 2.7M+ generation records across 42 sites

The model learns "what should this site generate?" — enabling deviation detection when actual output falls significantly below expectation.



SOLARPULSE



Model Training & Evaluation

Training Split

80%

Test Split

20%

MAE

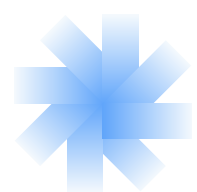
**Avg Abs
Error**

RMSE

**Penalizes
Large Errors**

R² Score

**Variance
Explained**



SOLARPULSE



RANDOM FOREST

Model Performance

MAE

(Mean Absolute Error)

:

2.3534

RMSE

(Root Mean Squared Error)

:

4.8842

R²

(Coefficient of Determination)

:

0.8420

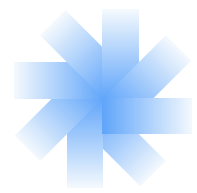
Model Performance

Random Forest vs Baseline

Baseline — MAE: 6.9220 | RMSE: 12.3342 | R²: -0.0077

Random Forest — MAE: 2.3534 | RMSE: 4.8842 | R²: 0.8420

RMSE reduced by ~60%. The model explains 84.2% of variation in solar generation on the held-out test set, confirming strong predictive accuracy across sites and weather conditions.



SolarPulse

Model Configuration & Validation

01

RF Core Settings: `n_estimators=100` trees; `max_depth=20` controls tree complexity and model capacity.

02

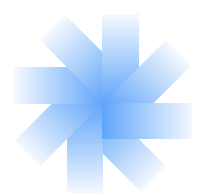
RF Regularization: `min_samples_leaf=2` limits overfitting; `random_state=42` ensures reproducibility; `n_jobs=-1` uses all CPU cores.

03

Warning Threshold $\approx -57.42\%$: separates normal variation from moderate negative deviations requiring attention.

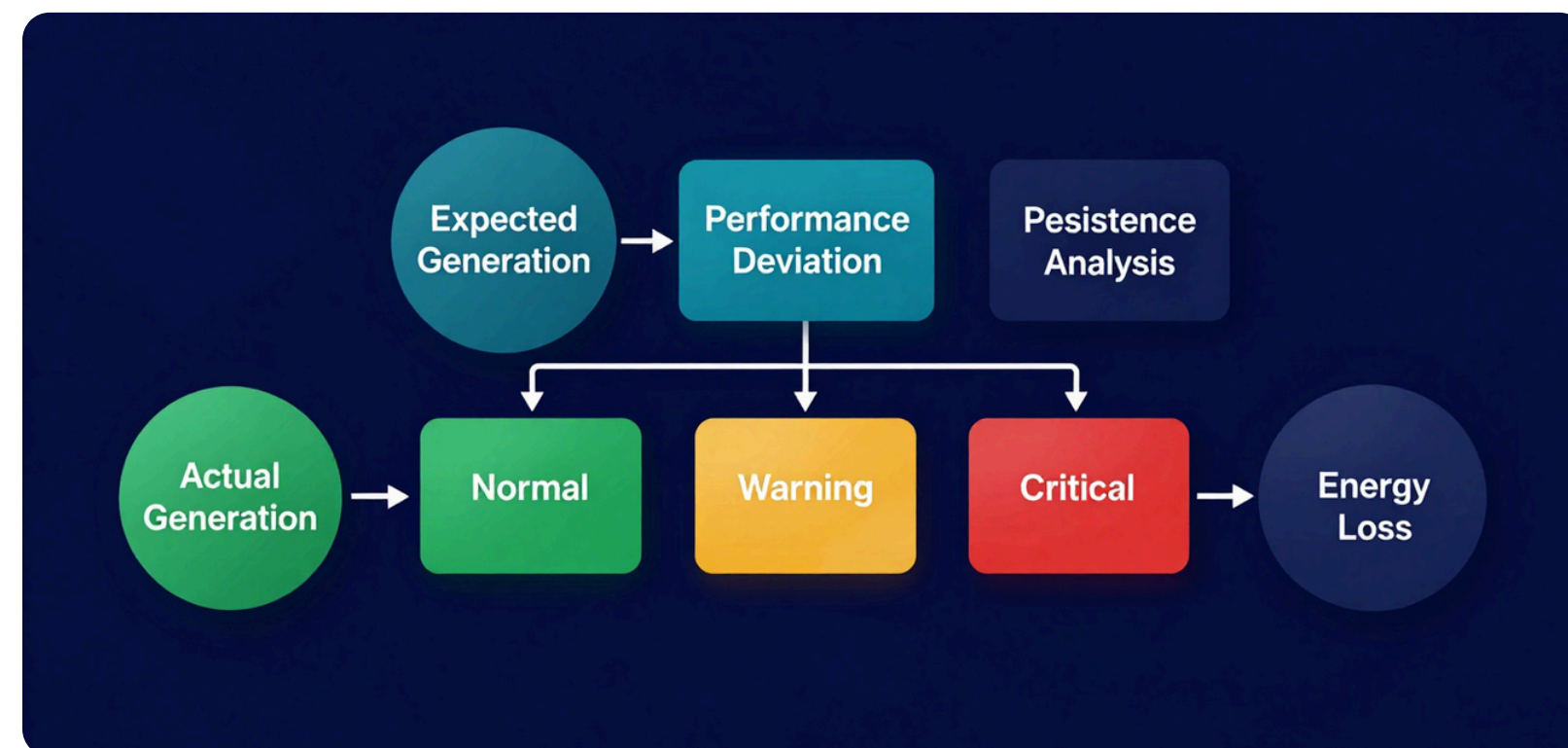
15

Critical Threshold $\approx -69.57\%$: flags severe underperformance; four consecutive critical readings trigger persistent underperformance alert.



SOLARPULSE

From Prediction to Underperformance Detection



PERFORMANCE DEVIATION FORMULA

$\text{Deviation \%} = (\text{Actual} - \text{Expected}) / \text{Expected} \times 100$

- Positive: Actual exceeds expected output
- Negative: Actual falls below expected output
- Floor protection: Ignored when Expected < 1.0 kW

CLASSIFICATION THRESHOLDS

- Normal: Deviation > -57.42%
- Warning: -69.57% to -57.42%
- Critical: $\leq -69.57\%$

Thresholds calibrated on validation period to separate normal operational variation from significant negative deviations.

PERSISTENCE ANALYSIS

A single critical observation → Critical Anomaly flagged.

Four consecutive critical 15-min readings (~1 hour) → Persistent Underperformance event recorded. Persistence logic filters transient noise and identifies sustained operational issues requiring investigation.

ENERGY LOSS ESTIMATION

$\text{Energy Loss} = \text{Expected} - \text{Actual}$

(applied when Actual < Expected for flagged observations)

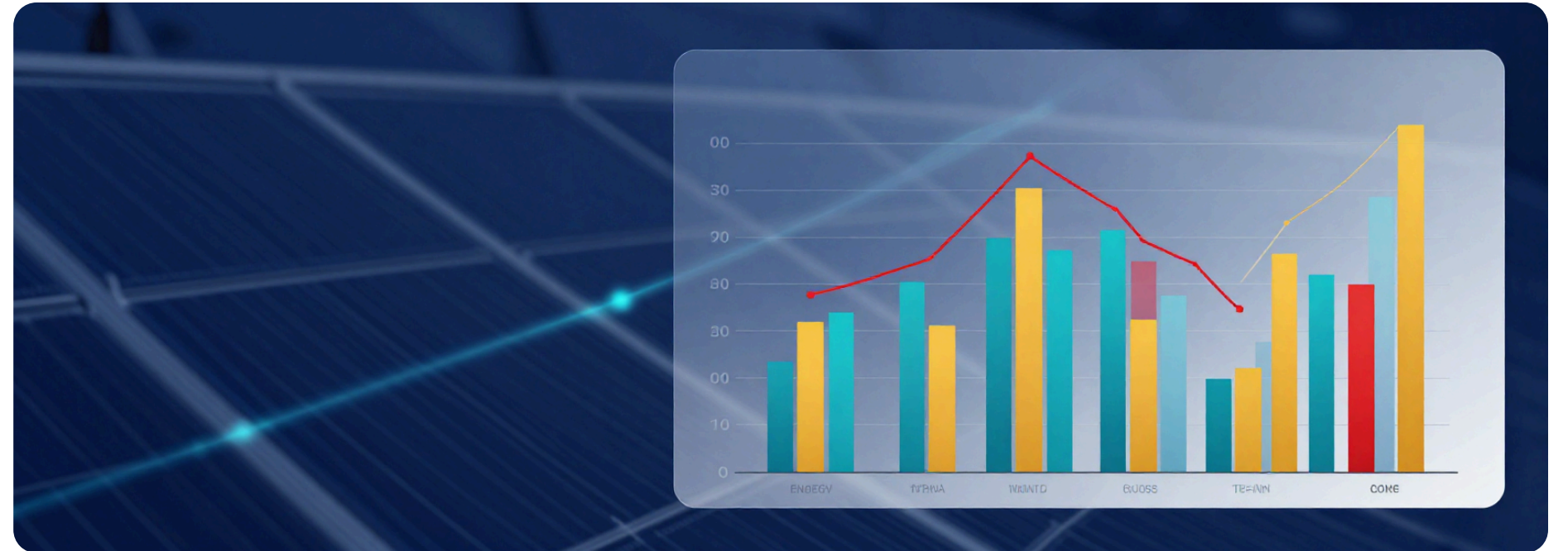
Lower-than-expected generation → Potential energy loss quantified → Sites prioritised → Maintenance or investigation triggered.

"Site 27 was identified as requiring investigation due to persistent lower-than-expected generation."



SOLARPULSE

Operational Intelligence



Energy Loss Formula

Expected –
Actual

Site Prioritization

Persistent
Events

Example Finding

Site 27: persistent lower-than-expected generation flagged for investigation

System Role

Supports investigation — does not prove physical failure



SOLARPULSE

Saved Project Outputs

Artifacts Saved for Pipeline Reuse

- solar_generation_model.pkl — Trained Random Forest model serialized for reuse in Streamlit and batch scoring.
- detector_config.json — Performance floor, Warning threshold ($\approx -57.42\%$) and Critical threshold ($\approx -69.57\%$) saved for consistent detection across all tools.
- ml_data_final.pkl — Final cleaned and feature-engineered ML dataset preserved for reproducibility.
- performance_results — Test-period actual vs expected generation and performance classification stored to MySQL.

```
processed
├── feature_importance.csv
├── ml_data_final.pkl
├── ml_data.pkl
├── Monthly_Summary_Solar.csv
├── Solar_Energy_Generation.csv
├── Solar_Site_Details.csv
├── Weather_Data_reordered_all.csv
└── ml
    ├── detector_config.json
    └── solar_generation_model.pkl
```

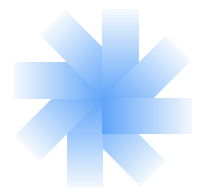
SQL Views for Power BI

- vw_solar_performance — Row-level actual vs expected generation, performance deviation %, and underperformance classification; primary source for Power BI site and time-series visuals.
- vw_site_performance — Site-aggregated summary of performance status, persistent events, and estimated energy loss; drives Executive Overview and Underperformance Monitor pages.

Pipeline Flow

Jupyter / Python → model + config + results → MySQL + file artifacts → Power BI + Streamlit

Saving artifacts ensures all tools share identical model logic and detection thresholds, guaranteeing consistency between notebook, database, dashboard and app.



SOLARPULSE

Process 5

Power BI Dashboard

Power BI connects directly to MySQL analytical views — vw_solar_performance and vw_site_performance — rather than rebuilding ML logic. This separates computation from reporting and ensures consistent results across all tools.

Data Flow:

MySQL → vw_solar_performance + vw_site_performance → Power BI

Four Dashboard Pages:

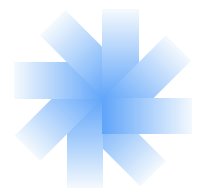
- Executive Overview — portfolio-level KPIs and trends
- Site Performance — individual site deep-dive
- Underperformance Monitor — prioritize sites needing attention
- ML Insights — model transparency and detection logic



Key Capabilities:

- Synchronized campus, site and date-range filters across all pages
- Actual vs Expected generation trend visualizations
- Performance deviation tracking and status classification
- Estimated energy loss quantification by site
- Persistent underperformance event counts
- Site risk ranking and investigation lists
- Random Forest model metrics (R^2 , MAE, RMSE)
- Feature importance and residual distribution charts

Power BI consumes pre-computed results — no ML re-execution required at report time.



Executive Overview

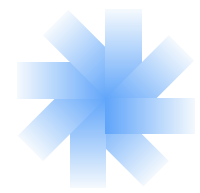


The Executive Overview provides a portfolio-level summary of all 42 solar sites. Key metrics include Actual Generation, Expected Generation, Overall Performance deviation, and Estimated Energy Loss. The Actual vs Expected trend chart reveals generation gaps over time, while the site performance ranking highlights top and bottom performers.

Filters: Campus, Site, Date Range — synchronized across all visuals for seamless drill-down.

"How is the overall solar portfolio performing?"

Performance status cards classify sites as Normal, Warning, or Critical based on SolarPulse thresholds. The investigation table surfaces sites with persistent underperformance and significant estimated energy loss — enabling rapid prioritization without manual data review. All visuals connect to MySQL views vw_solar_performance and vw_site_performance, ensuring ML results drive every insight.



SOLARPULSE — Power BI Dashboard

Power BI — Site Performance



Provides detailed investigation of individual solar sites. Key metrics include Actual Generation, Expected Generation, Performance Deviation %, and Site Information (capacity, panels, inverter). Filters for Campus, Site, and Date Range allow focused drill-down.

Answers: "How is this particular site performing compared with SolarPulse expectations?"

Visualizations include Actual vs Expected Generation trend over time, Performance Deviation over time, and a Performance Events table listing warning and critical observations. Performance status indicators highlight sites requiring investigation, enabling targeted maintenance prioritization.



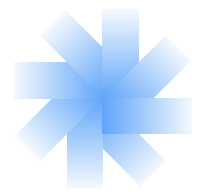
Underperformance Monitor



The Underperformance Monitor answers: "Which sites need attention first, and where is the greatest potential energy loss?"

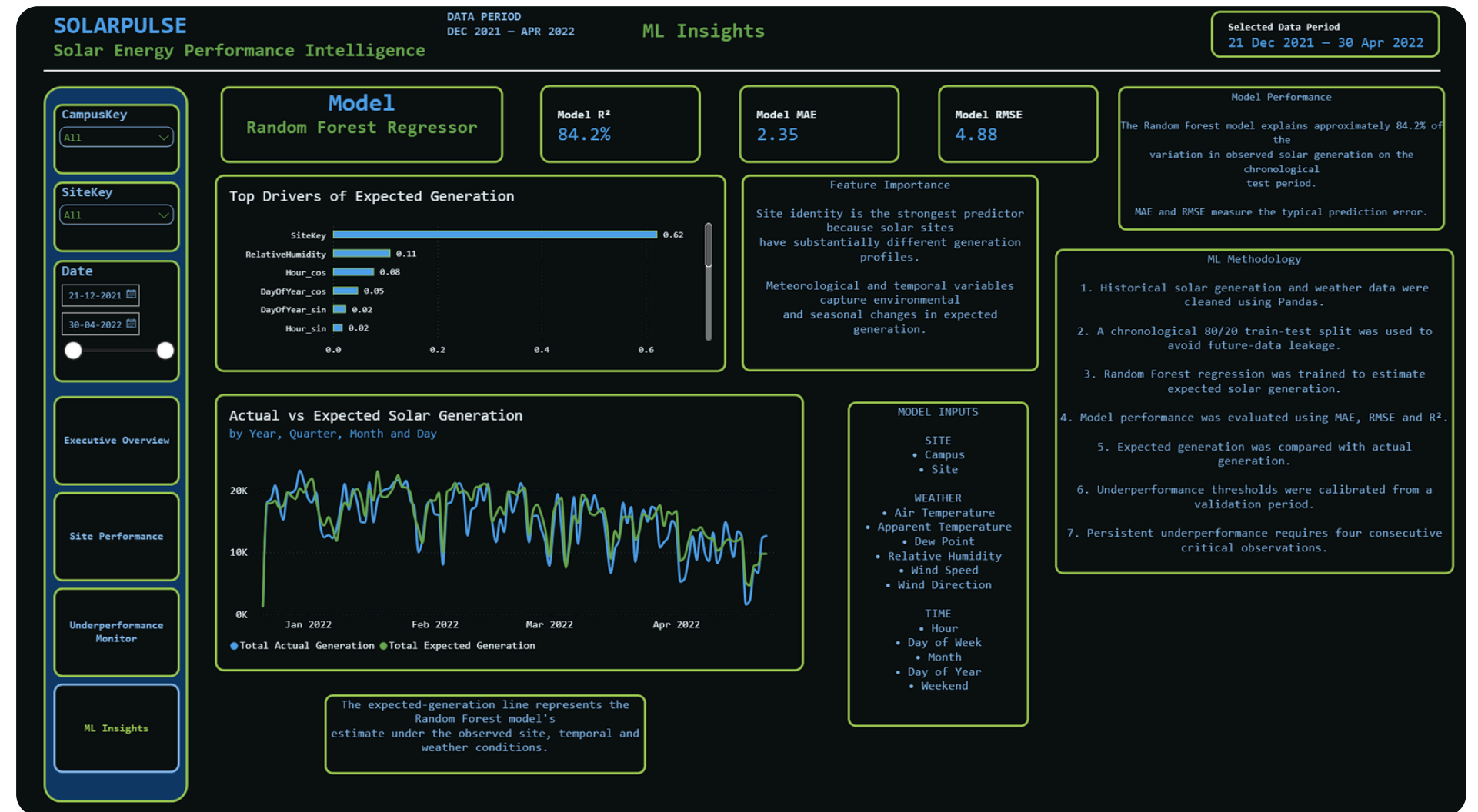
- Critical Events Count — total critical anomaly observations across all sites
- Persistent Underperformance Events — sequences of ≥ 4 consecutive critical 15-min readings (~1 hour)
- Potential Energy Loss by Site — estimated kWh gap where actual < expected
- Sites at Risk — ranked list highlighting highest-priority sites for investigation

- Operational Status Distribution — breakdown of Normal / Warning / Critical observations across the portfolio
 - Site Risk Ranking — ordered by persistent event count and energy loss magnitude
 - Persistent Events by Site — bar chart comparing underperformance frequency per site
 - Investigation List — actionable table of sites flagged for maintenance review
- Filters: Campus · Site · Date Range. Synchronized across all dashboard pages.



SolarPulse Analytics

Power BI — ML Insights



MODEL TRANSPARENCY

Random Forest Regression — 100 trees, max depth 20, min samples per leaf 2, reproducible seed 42, all CPU cores utilised.

PERFORMANCE METRICS

- $R^2 = 0.8420$ — model explains 84.2% of generation variance on held-out test data
- MAE = 2.3534 kWh — average absolute prediction error
- RMSE = 4.8842 kWh — penalises larger deviations
- Baseline $R^2 = -0.0077$ (mean predictor) — confirms strong model lift

DETECTION THRESHOLDS

- Warning threshold $\approx -57.42\%$ deviation
- Critical threshold $\approx -69.57\%$ deviation
- Performance floor: expected < 1.0 kWh excluded from classification

TOP FEATURES (by importance)

1. Hour_sin / Hour_cos — time-of-day cyclical encoding
2. DayOfYear_sin / DayOfYear_cos — seasonal position
3. AirTemperature & ApparentTemperature
4. DewPointTemperature & RelativeHumidity
5. SiteKey — site-specific generation capacity
6. WindSpeed & wind direction components

METHODOLOGY SUMMARY

Chronological 80/20 train-test split prevents future-to-past data leakage. Model trained exclusively on observed (non-missing) SolarGeneration targets. Thresholds calibrated on a held-out validation window within the training period. Residual analysis confirms no systematic bias across sites or seasons.

"SolarPulse predictions are grounded in site conditions, weather, and time — not assumptions."



Process 6 — Streamlit Prediction App

PURPOSE

Interactive expected-generation prediction interface powered by the saved Random Forest model — no retraining required at runtime.

USER INPUTS

- Campus & Solar Site
- Date & Time
- Air Temperature & Apparent Temperature
- Dew Point & Relative Humidity
- Wind Speed & Wind Direction

Automatic feature engineering converts raw inputs into cyclical time features, wind components, and site identifiers before passing to the model.

PREDICTION FLOW

USER INPUT



FEATURE ENGINEERING

(Hour_sin/cos, DayOfYear_sin/cos, WindDirection_sin/cos, Month, DayOfWeek, IsWeekend)



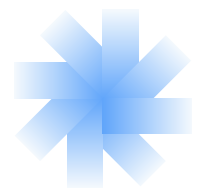
SAVED RANDOM FOREST

(solar_generation_model.pkl)



EXPECTED ENERGY GENERATION

The app loads the pre-trained model and detector_config.json thresholds, ensuring predictions and classifications are fully consistent with the notebook, database, and Power BI outputs.



SolarPulse

Prediction + Performance Assessment

01

Mode 1 — Prediction: "What should this site generate under these conditions?"

02

Mode 2 — Performance Assessment: "Is actual output significantly below expectation?" Deviation = $(\text{Actual} - \text{Expected}) / \text{Expected} \times 100$.

03

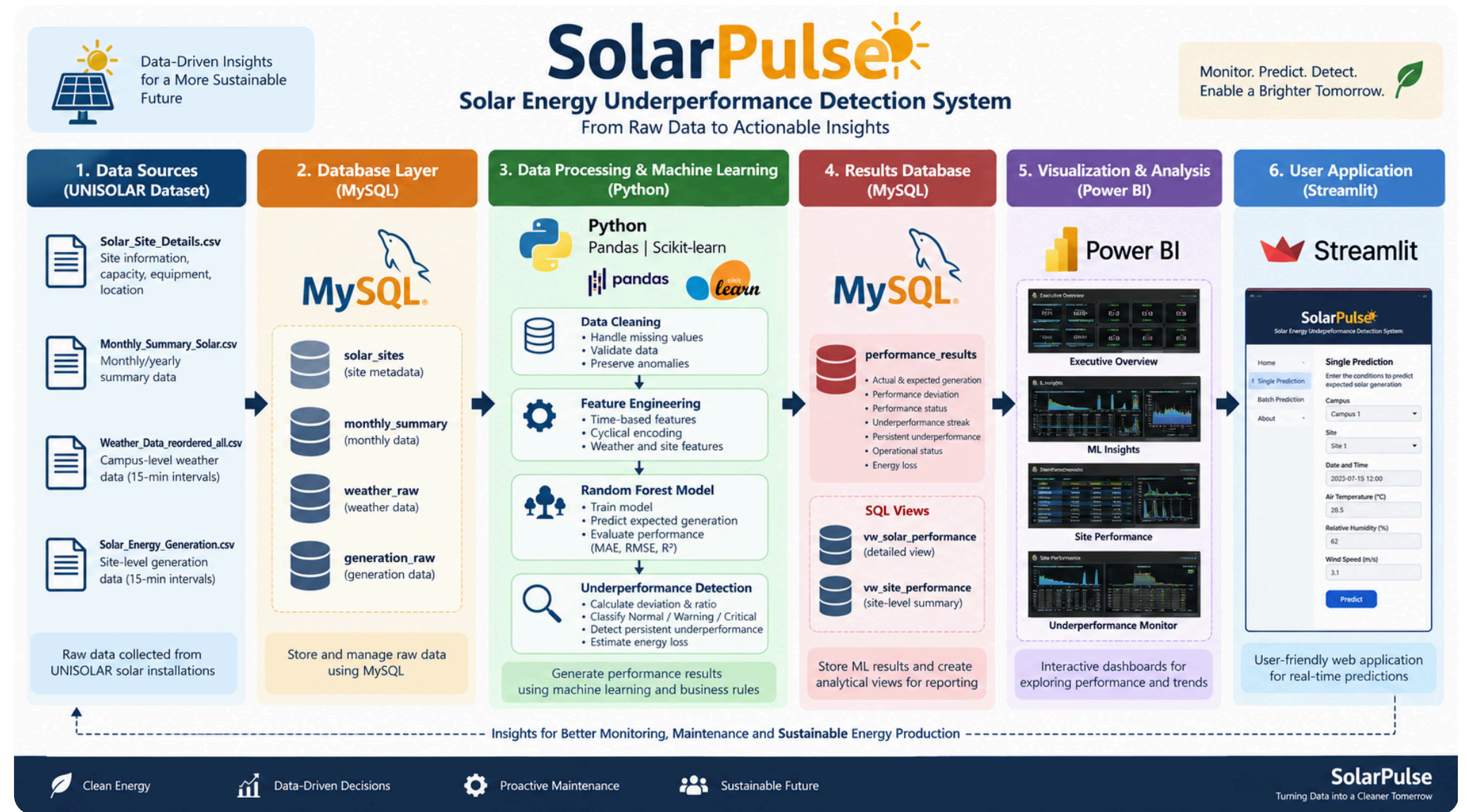
Threshold Classification: Normal (above $\approx -57.42\%$), Warning ($\approx -69.57\%$ to -57.42%), Critical ($\leq \approx -69.57\%$).

04

Shared thresholds ensure consistency across notebook, database, Power BI and Streamlit app.



End-to-End Architecture

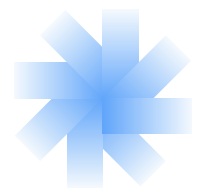


Pipeline Stages

① UNISOLAR CSV Files → ② Python Data Ingestion → ③ MySQL Raw Database → ④ SQL Data Quality Audit → ⑤ Python/Pandas Cleaning → ⑥ Feature Engineering → ⑦ Machine Learning (Random Forest) → ⑧ Expected Generation → ⑨ Underperformance Detection → ⑩ MySQL Analytical Results → ⑪ Power BI Monitoring + Streamlit Prediction App

Component Roles

- 🗄️ SQL — Structured storage and querying of all solar, weather and site data
- 🐍 Python/Pandas — ETL, cleaning, transformation and feature engineering
- 🤖 ML — Random Forest prediction and underperformance anomaly logic
- 📊 Power BI — Interactive monitoring dashboard and business insights
- ⚡ Streamlit — Interactive expected-generation prediction and performance assessment app



SOLARPULSE

Future Scope & Roadmap

Real-Time Weather API

01

Live Inverter / SCADA
Integration

02

Automated
Underperformance
Alerts

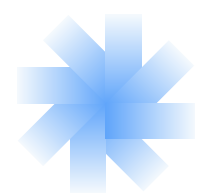
03

Advanced Forecasting

04

Cloud Deployment

05



SOLARPULSE

Conclusion

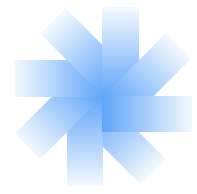


MySQL scalable storage ingesting and managing more than 3.1 million raw records across four structured tables.

SQL data-quality audit and Python/Pandas cleaning with targeted imputation and full feature engineering.

Random Forest ML model achieving R^2 0.84 for expected-generation prediction and threshold-based underperformance detection.

Power BI interactive monitoring dashboard and Streamlit prediction app delivering end-to-end operational intelligence.



SOLARPULSE

Thank You

Presented by: Aditya Ujjwal

Solar Energy Underperformance
Detection System